EventDeck

SRM INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

MCA PROJECT REVIEW · OCTOBER 2026

EVENTDECK: A UNIVERSITY EVENT TICKETING SYSTEM WITH TRANSACTIONAL SEAT RESERVATION AND QR CHECK-IN

SUBMITTED BY

MATHESHWARAN R
RA2532241010311

SUHAS C
RA2532241010312

NITHYASHREE K
RA2532241010329

DEVANANDA E
RA2532241010332

UNDER THE GUIDANCE OF

Dr. V. Nisha, M.Sc., M.Phil., PhD.
Assistant Professor, Department of Computer Applications

Master of Computer Applications · Department of Computer Applications · Faculty of Science and Humanities

SRM Institute of Science & Technology, Kattankulathur – 603 203, Chennai, Tamil Nadu

ABSTRACT

One system from event publication to admission

EventDeck is a web-based event management and ticketing system for a multi-campus university. Built with Next.js, React, TypeScript and Supabase PostgreSQL, it lets students, faculty, alumni and visitors from other colleges discover campus events, book free or paid tickets and receive QR-coded tickets. Organizers publish events with ticket tiers, seat limits, registration and check-in windows, and audience rules for campuses, institutions or participant categories. Seats are reserved inside database transactions, unpaid seats are held for ten minutes, each person may hold at most four tickets per event, and Razorpay test-mode payments are verified on the server. At the venue, organizers and appointed moderators scan tickets, and each ticket is admitted only once. Administrators review organizer applications, cancellations, refunds and payouts and export reports.



Problem

Forms, spreadsheets and payment screenshots cannot enforce seats, eligibility or single entry



Approach

Server-authoritative rules, database transactions and constraints, row-level security



Capabilities

Tiers and audience rules, holds, verified payments, QR tickets, moderators, refunds, payouts



Outcome

No overselling, trustworthy payments, and an auditable record of every booking

Campus events outgrew forms and screenshots

Domain: event management for a multi-campus university. SRM has students, faculty, staff and alumni across Kattankulathur, Ramapuram and Vadapalani, and many events welcome other colleges and the public.

TODAY

- Seat limits are not checked when a form is submitted, so events get overbooked
- Payments are verified from screenshots that can be edited or reused
- Campus or college eligibility depends on what applicants type
- Passes can be copied and shown twice at different gates

PROBLEM STATEMENT

Design and build a university ticketing system that never issues more tickets than seats, applies audience rules from trusted profiles, verifies payments with the provider, and admits each ticket exactly once, even when several moderators scan at the same time.

Why it matters: fairness for attendees, trustworthy money handling, and safe, fast entry at the door.

What we set out to build, and where it stops

Primary objectives

- Verified onboarding for SRM campuses, other colleges and guests; organizer access by admin approval
- Events with tiers, windows and audience rules
- Transactional booking: atomic seats, 4 per person, 10-minute holds
- Server-verified payments, QR tickets and PDF passes
- Event-scoped moderators; each ticket admitted once

Core functional scope

- Participant, organizer, moderator and administrator workflows
- Discovery, full-text search and poster-led home page
- Cancellations with refunds; organizer payouts
- In-app realtime notifications and audit log
- CSV reports for organizers and administrators

Boundaries

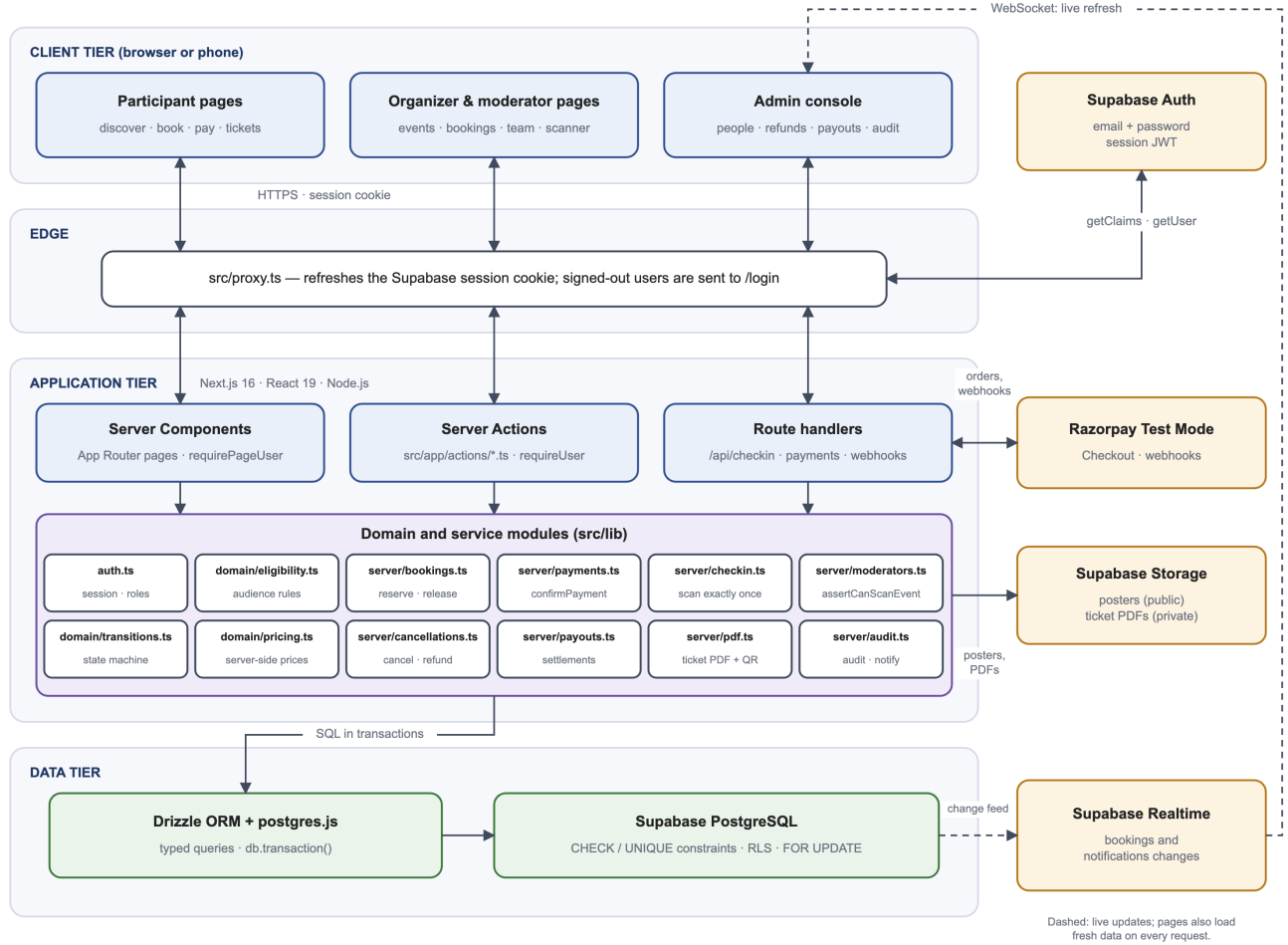
- Razorpay Test Mode only; live keys are refused
- Payouts are recorded; the bank transfer happens outside the system
- Not included: waiting lists, invite-only lists, numbered seat maps, native mobile apps

EXISTING SYSTEM VS PROPOSED SYSTEM

From manual checks to rules the database enforces

ASPECT	EXISTING · forms, sheets, screenshots	PROPOSED · EventDeck
Seat limits	Not checked; reconciled by hand afterwards	Atomic conditional update inside a transaction
Payments	Screenshots checked by hand	Razorpay signature verified on the server; webhooks deduplicated
Eligibility	Self-declared in a form	Checked on the server from the stored profile
Entry	Printed lists, copyable passes	Random 192-bit QR token; the first scan wins
Door staff	Access to the whole response sheet	Event-scoped moderators see only the scan result
Cancellations	Informal; refunds untracked	Admin-approved; tickets invalidated; refunds tracked
Visibility	No live view	Dashboards, realtime notifications, audit log

Four tiers, with rules kept on the server



Client & edge

Browser or phone. src/proxy.ts refreshes the Supabase session and redirects signed-out users.

Application tier

Server Components, Server Actions and route handlers call shared domain modules in src/lib.

Data tier

Supabase PostgreSQL through Drizzle ORM; constraints and row-level security as the final guard.

External services

Supabase Auth, Storage and Realtime; Razorpay Test Mode via server routes and signed webhooks.

Modern, typed and fully managed

Framework

Next.js 16.4 (App Router)
React 19 · TypeScript 5

Interface

Tailwind CSS 4
shadcn/ui on Base UI

Database

Supabase PostgreSQL
Drizzle ORM 0.45 · postgres.js

Auth & e-mail

Supabase Auth sessions
Brevo SMTP for auth e-mails

Payments

Razorpay Test Mode
HMAC-verified checkout & webhooks

Tickets

qrcode · pdf-lib
html5-qrcode scanner · sharp

Testing

Vitest (unit, integration)
Playwright (end-to-end)

Hosting

Vercel · eventdeck.online
Vercel Cron for hold expiry

One account, responsibilities that change with context

Participant

Discovers and searches events, books free or paid tickets or a pass, pays, downloads QR tickets and PDF passes, applies to become an organizer.

Organizer

Creates and publishes events with tiers, windows and audience rules; reviews pass requests; appoints moderators; scans; views bookings and attendance; requests payouts.

Event moderator

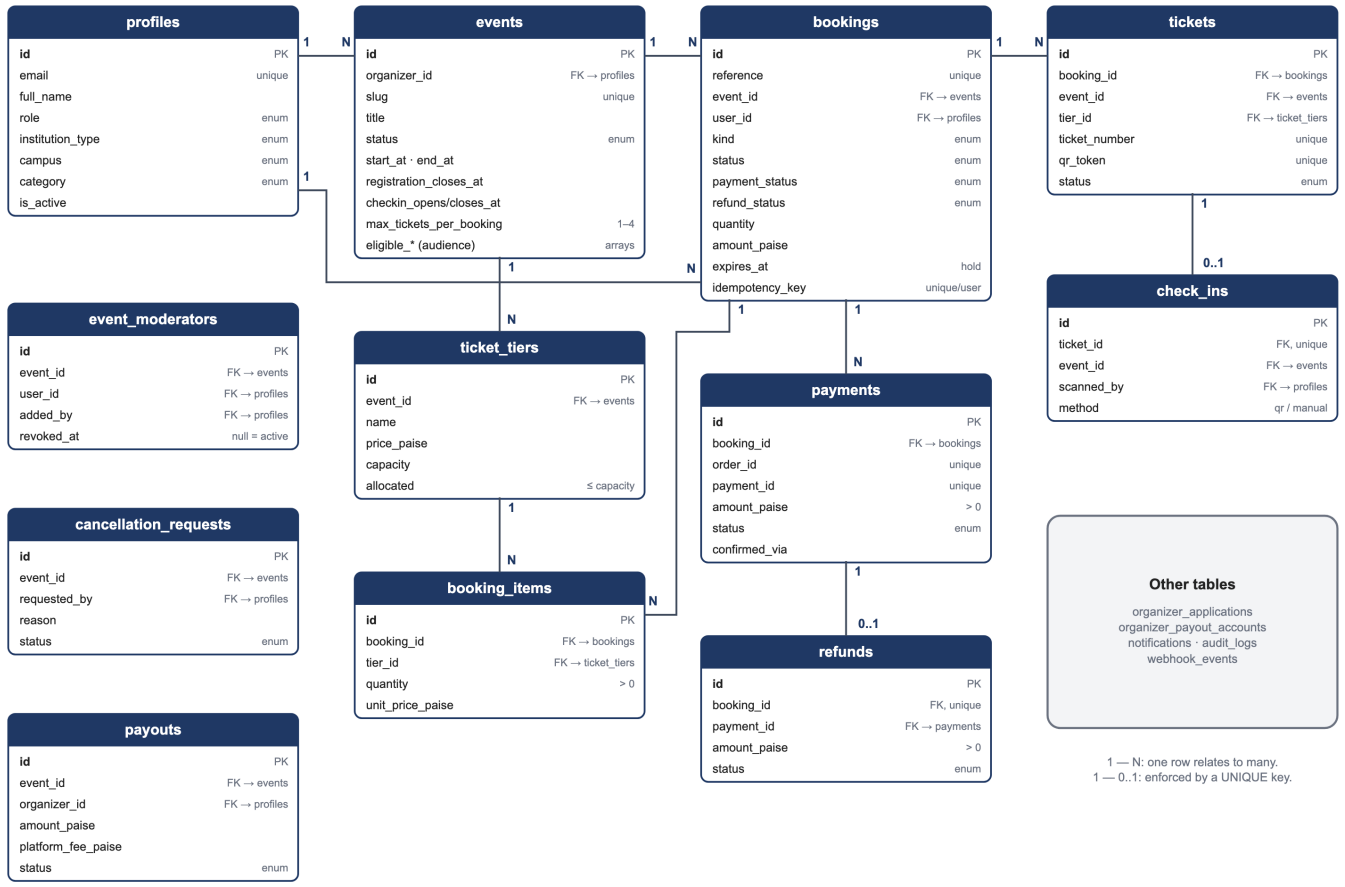
An ordinary account assigned to one event (up to 10 per event). Sees only the scanner for that event; checked on every scan.

Administrator

Approves organizers, cancellations and payouts; processes refunds; resolves payment exceptions; reads the audit log; exports reports.

Moderators are an assignment, not a role. A student who organizes one event can still book tickets for others and moderate a friend's event from the same account.

17 tables, constraints and row-level security



Core chain

profiles → events → ticket_tiers; bookings → booking_items, tickets → check_ins

Separate status fields

Booking, payment, refund and ticket status are stored independently

Constraints as a safety net

allocated ≤ capacity · 1–4 tickets per person · one check-in per ticket · unique idempotency key

Row-level security

Enabled on all 17 tables; the browser key cannot write, so every write goes through the server

Seats are taken in one transaction, or not at all

1

Idempotency

A repeated submission returns the original booking

2

Release holds

Unpaid seats older than 10 minutes are freed

3

Lock & validate

Profile row locked; event open; eligibility; ≤ 4 tickets

4

Reserve atomically

Conditional update succeeds only if seats remain

5

Create booking

Free → approved
Paid → 10-min hold
Pass → organizer review

THE RULE THAT PREVENTS OVERSELLING

```
UPDATE ticket_tiers
   SET allocated = allocated + n
 WHERE id = :tier
   AND allocated + n <= capacity;
```

PAYMENT PATH

- Razorpay order created from the booking amount (never from the browser)
- Checkout callback and webhook verify the HMAC signature; first one confirms, the second is ignored
- Late payment re-takes the seats, or becomes a payment exception for an admin

Every ticket admitted once; every request checked twice

CHECK-IN AT THE VENUE

Authorise the scanner

Admin, owning organizer, or an active moderator of this event

Parse the QR

Contains only EVD1: plus a 192-bit random token; no personal data

Validate from the database

Right event, valid ticket, approved booking, payment, inside the check-in window

Record once

INSERT … ON CONFLICT (ticket_id) DO NOTHING; the first scan wins on any device

DEFENCE IN DEPTH

Edge proxy refreshes session; redirects signed-out users

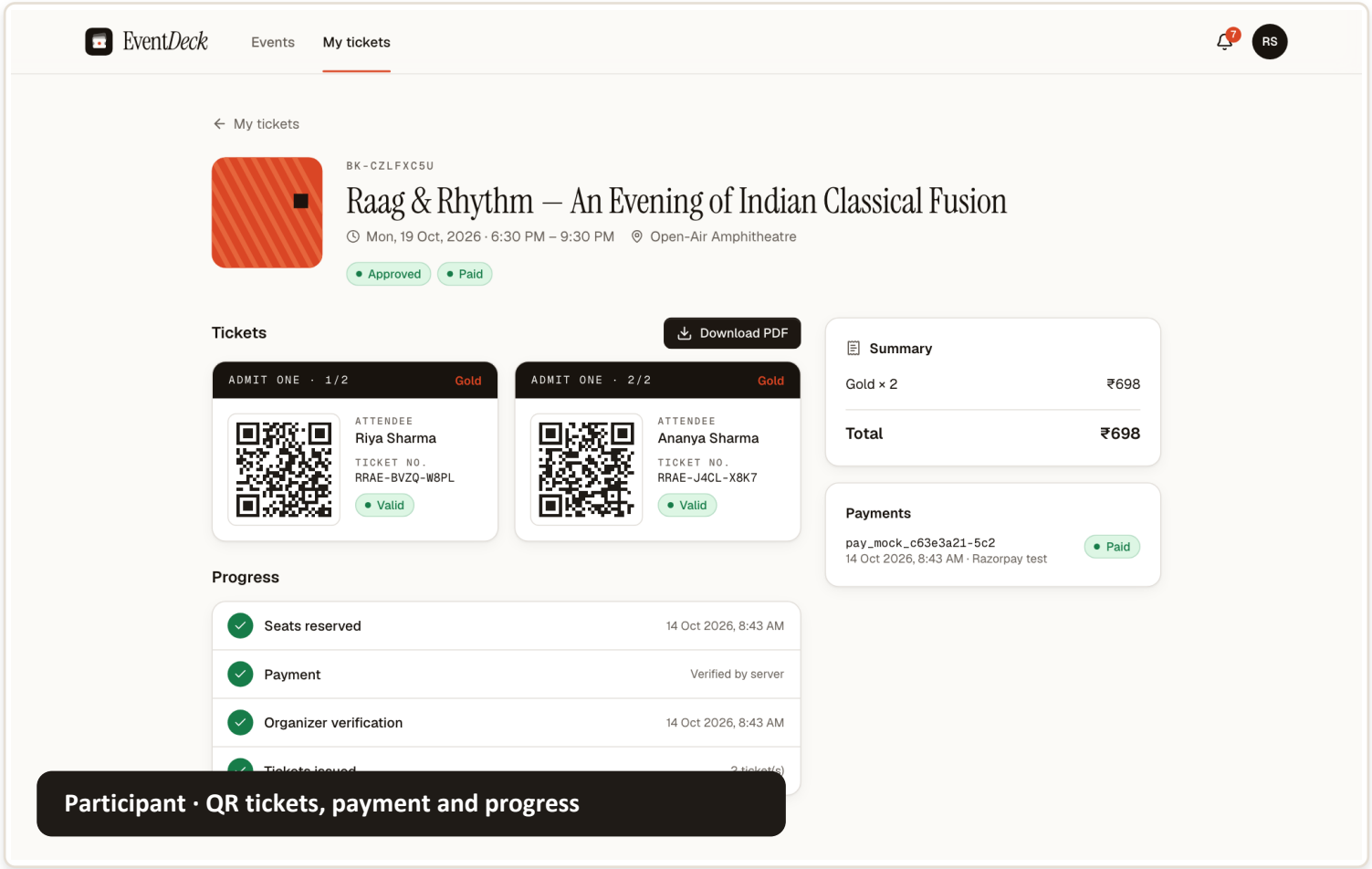
Role gate requirePageUser / requireUser on every page and action

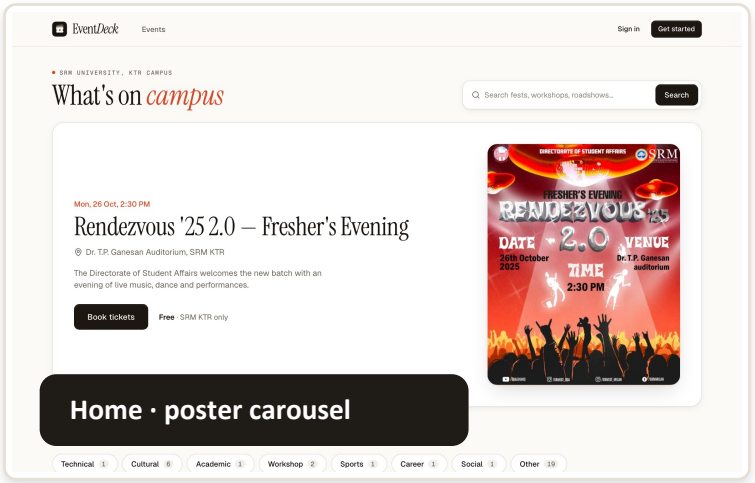
Resource checks assertCanManageEvent · assertCanScanEvent · ownership

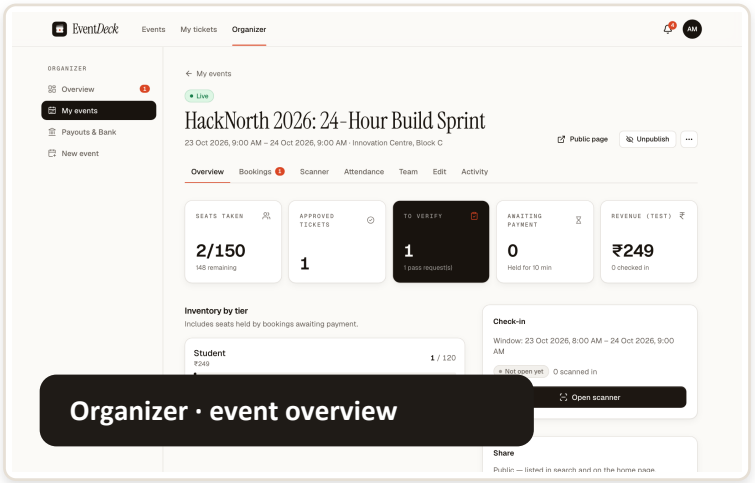
Database constraints + row-level security on all tables

Never trusted from the client: role, price, per-person limit, audience fields, payment success, ticket validity.

The working application, captured with demo data







Screens captured from the running application on 15 Oct 2026 using fictional demonstration accounts.

Evidence, reported exactly as executed

63 / 63

unit tests passed

15 Oct 2026, current code

0

type and lint errors

tsc --noEmit · ESLint

46

routes built

production build succeeded

39 / 39

ticket tiers consistent

allocated = seats in live bookings

CONCURRENCY & ACCESS-CONTROL EVIDENCE · QA RUN, 14 OCT 2026

- 94 / 94 Vitest tests (41 unit + 53 integration) and 26 / 26 Playwright runs passed
- 20 concurrent buyers for 5 seats: exactly 5 bookings succeeded
- 4 simultaneous scans by two moderators: exactly 1 check-in
- Checkout callback racing two webhooks: exactly 1 confirmation
- QA found and fixed 2 critical defects that let users claim SRM membership

Not yet re-run on current code

The 53 integration and 24 end-to-end tests need a dedicated Supabase test project, not yet created. The 14 Oct results above came from an earlier local stack that has since been removed.

The rules of an event belong in the server and the database

CONCLUSION

- EventDeck enforces seat limits, audience rules, per-person limits and single admission in the server and the database, not by hand.
- Organizers, moderators and administrators get the tools a university needs, from publication and payments to refunds, payouts and audit.
- Testing against a real database exposed defects that unit tests alone would have missed.

FUTURE ENHANCEMENTS

- Re-run integration and E2E suites on a dedicated test project
- Waiting lists for sold-out tiers
- Invite-only events with named guest lists
- Live payments and automated payouts, after review
- Load testing and a pilot with real organizers
- Offline-tolerant scanning for door devices

EventDeck

Thank you

Questions and discussion

LIVE APPLICATION

eventdeck.online



TEAM

Matheshwaran R · Suhas C · Nithyashree K · Devananda E

Guide: Dr. V. Nisha, Assistant Professor, Department of Computer Applications